

1.b) Bank-to-Turn, Trajectory, and Simulation:

Figure 1: Turning radius plot

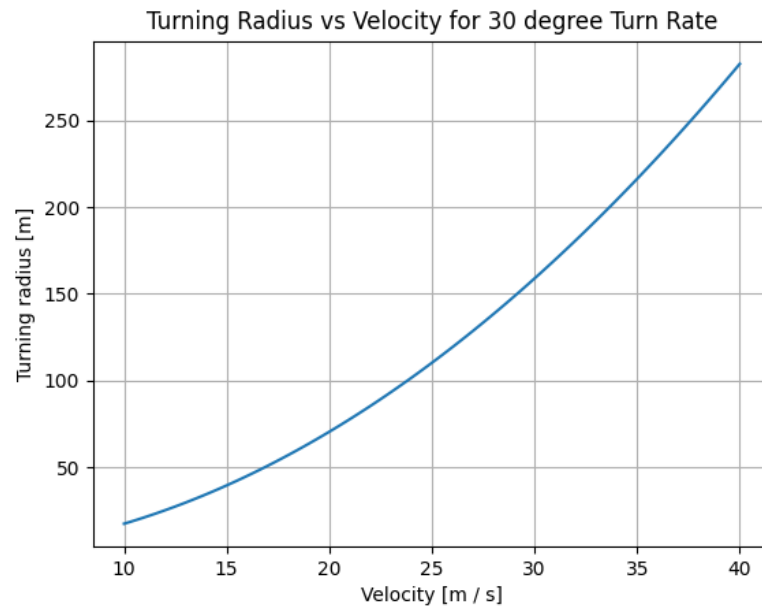


Figure 2: Reference trajectory plot

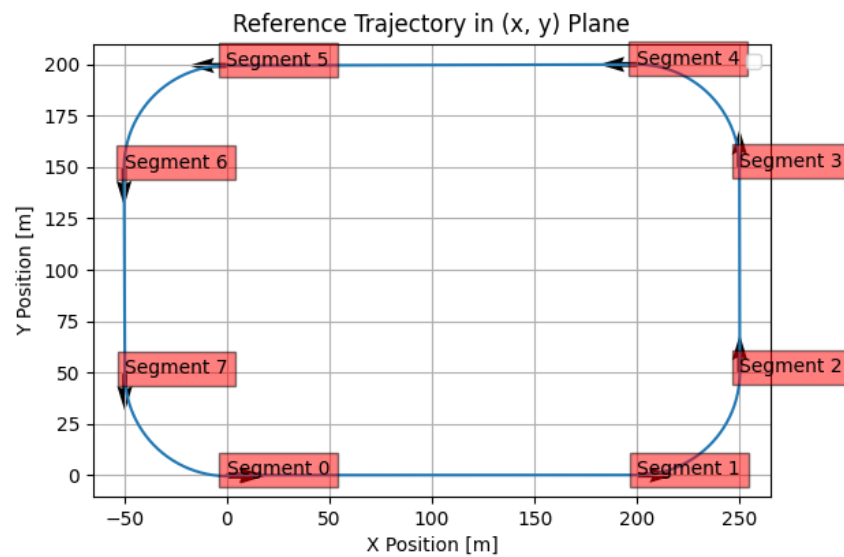


Figure 3: Feasibility check of turn required bank angles

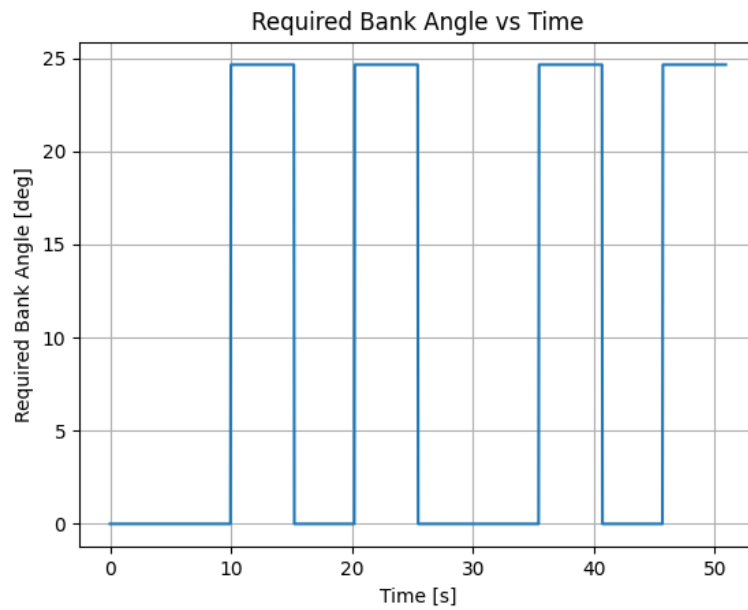
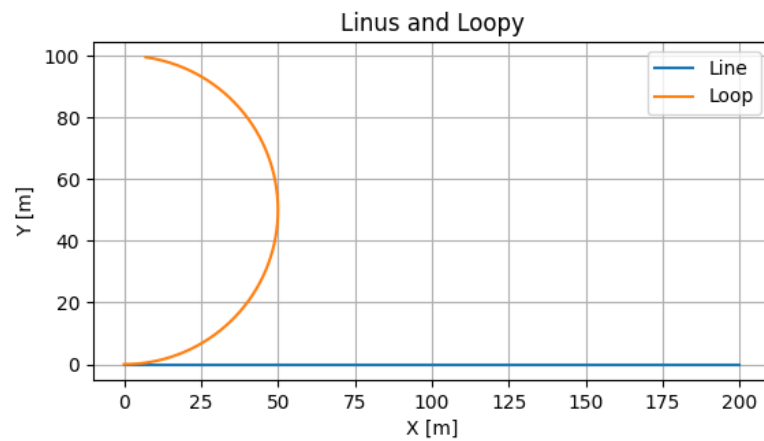


Figure 4: Plots of Linus and Loopy simulation



Code Snippet 1: SE(2) Fixed-Wing Velocity Kinematics Code

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from Code.SE2.SE2_maps import se2_compose, se2_exp, se2_log, se2_wedge, se2_vee, se2_inverse
4 from Code.SE2.SE2_integrators import se2_euler_integrator, se2_Lie_group_integrator
5

```

```

6  ## 1b.1) Plot turning radius vs velocity
7  def turning_radius(velocity, desired_turn_rate):
8      g = 9.81 # [m / s2]
9
10     turn_radius = velocity ** 2 / (g * np.tan(desired_turn_rate))
11
12     return turn_radius
13
14  V = np.linspace(10, 40, 100) # [m / s]
15  phi_0 = np.deg2rad(30) # Desired turn rate [rad]
16  turn_radius = turning_radius(V, phi_0)
17
18  plt.plot(V, turn_radius)
19  plt.xlabel("Velocity [m / s]")
20  plt.ylabel("Turning radius [m]")
21  plt.title("Turning Radius vs Velocity for 30 degree Turn Rate")
22  plt.grid()
23  plt.savefig("../HW3/Parts/Q1/AAE590LGM_HW3_Q1_turn_radius.png")
24  plt.show()
25
26  ## 1b.2) Propagate waypoints
27  X_0 = np.eye(3)
28
29  # Segment info
30  N_seg = 8
31  v_x = np.array([20, 15, 20, 15, 20, 15, 20, 15]) # [m / s]
32  v_y = np.zeros_like(v_x)
33  omega = np.array([0, 0.3, 0, 0.3, 0, 0.3, 0, 0.3]) # [rad / s]
34  T = np.array([10, 5.24, 5, 5.24, 10, 5.24, 5, 5.24]) # [s]
35
36  xi_k = np.array([[v_x], [v_y], [omega]])
37  dt = 0.04 # [s]
38
39  # Simulation loop
40  X_k = np.zeros([3, 3, N_seg + 1]) # Waypoint group elements
41  X_k[:, :, 0] = X_0
42
43  T_seg = np.cumsum(T)
44  time = np.arange(0, T_seg[-1], dt)
45  k_seg = np.round(T / dt).astype(int)
46  k_seg_total = sum(k_seg) + 1
47  X_seg = np.zeros([3, 3, k_seg_total])
48  X_seg[:, :, 0] = X_0
49  xi_seg = np.hstack((np.matlib.repmat(xi_k[:, :, 0], 1, k_seg[0]),
50                      np.matlib.repmat(xi_k[:, :, 1], 1, k_seg[1]),
51                      np.matlib.repmat(xi_k[:, :, 2], 1, k_seg[2]),
52                      np.matlib.repmat(xi_k[:, :, 3], 1, k_seg[3]),
53                      np.matlib.repmat(xi_k[:, :, 4], 1, k_seg[4]),
54                      np.matlib.repmat(xi_k[:, :, 5], 1, k_seg[5]),
55                      np.matlib.repmat(xi_k[:, :, 6], 1, k_seg[6]),
56                      np.matlib.repmat(xi_k[:, :, 7], 1, k_seg[7])))
57
58  for k in range(N_seg):
59      X_k[:, :, k + 1] = se2_Lie_group_integrator(X_k[:, :, k], xi_k[:, :, k], T[k])
60
61  for s in range(k_seg_total - 1):
62      X_seg[:, :, s + 1] = se2_Lie_group_integrator(X_seg[:, :, s], xi_seg[:, s], dt)
63
64  # Plot
65  # Trajectories
66  plt.plot(X_seg[0, 2, :], X_seg[1, 2, :])
67  plt.quiver(X_k[0, 2, :], X_k[1, 2, :], X_k[0, 0, :], X_k[1, 0, :])
68  for k in range(N_seg):
69      plt.text(X_k[0, 2, k], X_k[1, 2, k], f"Segment {k}", fontsize=10,
70              bbox=dict(facecolor='red', alpha=0.5))
71  plt.xlabel("X Position [m]")
72  plt.ylabel("Y Position [m]")
73  plt.title("Reference Trajectory in (x, y) Plane")

```

```

74 plt.legend()
75 plt.grid()
76 plt.gca().set_aspect('equal', adjustable='box')
77 plt.savefig("./HW3/Parts/Q1/AAE590LGM_HW3_Q1_reftraj.png")
78 plt.show()
79
80 def required_bank_angle(velocity, omega):
81     g = 9.81 # [m / s2]
82
83     bank_angle = np.atan(omega * velocity / g)
84
85     return bank_angle
86
87 bank_angles = required_bank_angle(xi_seg[0, :], xi_seg[2, :])
88 plt.plot(time[0:-1], np.rad2deg(bank_angles))
89 plt.xlabel("Time [s]")
90 plt.ylabel("Required Bank Angle [deg]")
91 plt.title("Required Bank Angle vs Time")
92 plt.grid()
93 plt.savefig("./HW3/Parts/Q1/AAE590LGM_HW3_Q1_feasck.png")
94 plt.show()
95
96 # Simulate loop and line
97 dt_loopleveline = 0.01 # [s]
98 tf = 10 # [s]
99 N_t = np.round(tf / dt_loopleveline).astype(int)
100 X_k_loop = np.zeros([3, 3, N_t + 1])
101 X_k_line = np.zeros_like(X_k_loop)
102 xi_line = np.array([[20], [0], [0]])
103 xi_loop = np.array([[15], [0], [0.3]])
104 X_k_loop[:, :, 0] = X_0
105 X_k_line[:, :, 0] = X_0
106
107 for k in range(N_t):
108     X_k_loop[:, :, k + 1] = se2_Lie_group_integrator(X_k_loop[:, :, k], xi_loop, dt_loopleveline)
109     a = se2_Lie_group_integrator(X_k_loop[:, :, k], xi_loop, dt)
110     X_k_line[:, :, k + 1] = se2_Lie_group_integrator(X_k_line[:, :, k], xi_line, dt_loopleveline)
111
112 plt.plot(X_k_line[0, 2, :], X_k_line[1, 2, :], label = "Line")
113 plt.plot(X_k_loop[0, 2, :], X_k_loop[1, 2, :], label = "Loop")
114 plt.xlabel("X [m]")
115 plt.ylabel("Y [m]")
116 plt.title("Linus and Loopy")
117 plt.legend()
118 plt.gca().set_aspect('equal', adjustable='box')
119 plt.grid()
120 plt.savefig("./HW3/Parts/Q1/AAE590LGM_HW3_Q1_linusloopy.png")
121 plt.show()

```